Synthetic Human Agent (`agent.pyw`)

Created by **Team Plankton & Ball** | NTU × ST Engineering Hackathon Submission

1. Executive Summary

This technical project documentation outlines the comprehensive architecture, design specifications, and forensic validation methodologies for the **Synthetic Human Agent** (`agent.pyw`). Developed by **Team Plankton & Ball** for the NTU × ST Engineering Hackathon, the agent operates continuously inside a secure Proxmox Windows virtual machine environment for over 17 minutes. It simulates realistic human productivity tasks—specifically Google Chrome web research across whitelisted domains, OpenOffice Writer text composition, and OpenOffice Calc data management—while ensuring absolute compliance with digital forensics auditing tools like Sysmon, Process Explorer, and Process Monitor.

2. Technical Architecture & Special Features

To bypass the rigid signatures and bulky overhead of third-party automation frameworks, Team Plankton & Ball engineered a lean, native execution model incorporating several advanced features:

- **Zero-Dependency Native `ctypes` Input Simulation:** Interfaces directly with Windows `user32.dll` through `SendInput` and virtual keycode maps, enabling precise, low-overhead mouse navigation, window state control, and keystroke injection without external binaries.
- **Clean `explorer.exe` Parent Process Ancestry:** Invokes desktop shell hooks (`os.startfile` and shell run dialogs) so that spawned applications (`chrome.exe`, `swriter.exe`, `scalc.exe`) inherit `explorer.exe` as their true Parent PID in Sysmon Event ID 1 logs, avoiding script-runner footprints.
- **Gaussian Timing Entropy Models:** Replaces rigid uniform sleep loops with Gaussian distributions (`random.gauss`) and weighted random variances to ensure organic, unpredictable time deltas between actions.
- **QWERTY Neighbor Typo Generation & Backspaces:** Implements realistic typing cadences that intentionally introduce neighboring key typos and execute natural backspace corrections before proceeding.
- **Automatic Window Maximization via `Win + Up`:** Automatically maximizes active application windows upon launch to mirror authentic user workspace organization.
- **Active Whitelisted Web Surfing & Office Automation:** Dynamically alternates between researching whitelisted professional domains, scrolling, searching (`Ctrl + F`), and authoring structured Office documents.

3. Scoring Criteria Alignment

Evaluation Criterion	Weight	Implementation Details by Team Plankton & Ball
Process Authenticity	30%	Shell hooks and `os.startfile` guarantee correct `explorer.exe` parentage, bypassing script runner footprints in Sysmon.
Human Entropy & Timing	30%	Gaussian variance models, variable inter-keystroke cadences, typos with backspaces, and reading pauses prevent robotic uniformity.
Complexity of Action	20%	Executes sophisticated multi-step workflows: whitelisted browsing, page scrolling, text queries, document authoring, and spreadsheet entry.
Resource Efficiency	20%	Minimalist pure Python design with clean application closures (`Alt + F4`) ensuring zero memory leaks or flatlined CPU usage.

4. Workflow Overview

The agent operates on an endless 17-minute loop, alternating between two primary simulation pipelines:

- **Web Research Simulation:** Launches Chrome, maximizes the window, and navigates exclusively across hackathon-approved whitelisted domains: `coinmarketcap.com`, `coursera.org`, `coinbase.com`, `coingecko.com`, `samsung.com`, `aws.amazon.com`, `amazon.com`, `apple.com`, `apnews.com`, and `dell.com`. Performs random scroll steps and active search queries (`Ctrl + F`).
- **Office Productivity Simulation:** Triggers OpenOffice Writer (`swriter`) and Calc (`scalc`) via shell dialogs, inputting technical infrastructure reports and structured metric tables with authentic keystroke noise and file saving routines.

5. Verification & Forensic Testing Methodology

Team Plankton & Ball validated the system using the official evaluation toolset: examining Sysmon Event ID 1 logs for clean `explorer.exe` parent mapping, analyzing Process Monitor (ProcMon) timestamp deltas for high timing variance, and monitoring Proxmox VM summary graphs for resource efficiency.